

EFES Master Node: a Full Custom FPGA plus ESP32 Implementation of the AUSB Acoustic Protocol

Gioele Giunta
s360501

Politecnico di Torino, LM 32 Computer Engineering
Course: Electronics for Embedded Systems

Academic Year 2025 / 2026

Note on this report. I proposed this project to the professor as an alternative to the written exam, and the professor approved the idea. So this is not a lab assignment: it is a personal project that I designed and built on my own, and what follows is the report I am sending to him. For this reason the text is written in the first person. The focus is the master node, that is the FPGA side plus the ESP32 that drives it.

Contents

1	Introduction	3
1.1	What this version is	3
1.2	The hardware I used	3
2	System overview	4
3	The FPGA SoC	4
3.1	Block diagram	5
3.2	The on chip bus and the address map	5
3.3	Arbitration	6
3.4	The simple slaves: UART and GPIO register maps	7
4	Clock tree	7
5	SDRAM controller	8
5.1	Bring up sequence	8
6	Audio accelerator and tone decoder	9
6.1	Timing in cycles and in seconds	10
6.2	The two state machines	11
6.3	FFT size and the tone bins	11
6.4	The tone decoder, with its checks	12
7	SPI master for the ADC, and the microphone front end	12
7.1	Why a custom SPI master and not a generic one	12
8	PWM speaker and LED	13

9	NOR flash controller	14
9.1	Memory layout	14
9.2	State machine	15
9.3	SCAN: finding the head after power up	16
9.4	Unlocking the chip and the polling timeout	16
9.5	Generic SPI engine register map	16
9.6	Note on the current architecture	16
10	Signal integrity and decoupling	17
10.1	Brownout during sector erase	17
10.2	SPI clock too fast for the wiring	17
10.3	Retry and verify in firmware	17
11	ESP32 firmware	18
11.1	Register level UART	18
11.2	Flash link: retry and verify, and a defensive read	18
12	The acoustic protocol	19
13	Power supply	19
14	What worked and what did not	19
15	Conclusions	19
A	Quick reference	20

1 Introduction

The AUSE protocol is a communication protocol that I created for my Bachelor thesis and later submitted to the IEEE IoT Magazine under the title *Design and Experimental Validation of a Low-Cost Acoustic Communication Protocol for Underground Sensor Networks*. The goal of the protocol is to let two devices talk to each other using only very cheap hardware, namely a speaker and a microphone. The protocol sends packets by emitting pairs of superimposed tones: at every step one tone identifies the logical channel (the carrier) and a second tone carries the data symbol.

On the receiver side an FFT moves the microphone signal from the time domain to the frequency domain, so that the pair of active frequencies can be read out as peaks in the spectrum. To keep the detection reliable in the presence of noise I had implemented, in the original thesis, several techniques: a machine learning stage that learns the noise floor and the low frequency roll-off of the cheap microphone (electret capsules lose sensitivity at low frequency), and a parabolic interpolation that refines the position of each spectral peak to better than one bin.

1.1 What this version is

The version documented here is a full custom FPGA implementation of the *master* side of my protocol, paired with an ESP32 in a hybrid arrangement. The FPGA is a Tang Nano 20K (Gowin GW2AR-18). I used Gowin EDA for synthesis, place and route and programming, and GHDL to simulate my VHDL before putting it on the board.

On the FPGA I preferred to write the interconnect, the peripherals and the memory controller myself in VHDL, and to drive their registers from a soft core CPU so that all the high level logic lives in software. I used only three IP cores from Gowin:

The **PicoRV32** soft core is the CPU that runs the firmware and drives every peripheral over the on chip bus. The **Gowin rPLL** takes the 27MHz board crystal and produces the two synchronous clocks the design needs (40.5 MHz for the bus and the CPU, and a phase shifted copy of it for the SDRAM die). The **Gowin FFT** core sits inside the hardware accelerator that I built around it to turn microphone samples into a spectrum.

Everything else on the FPGA is custom VHDL that I wrote: a generic UART module, a generic SPI byte engine used to talk to the NOR flash, a second SPI master specialised and timed for receiving audio from the ADC, a generic GPIO peripheral that switches the indicator LEDs, and a PWM peripheral that generates the output tones for the speaker.

On the ESP32 side, the board connects to the FPGA through two independent UART links, so four wires in total (two TX and two RX), accessed at register level. The ESP32 runs the AUSE protocol logic: it queues the inbound and outbound symbols, rebuilds the packets, and decides which packet to send next. The ESP32 is also connected to the internet and exchanges commands with a web dashboard over a WebSocket. In this arrangement the FPGA is the interface that lets the system speak my acoustic protocol, and the ESP32 is the brain that runs the protocol on top of it.

1.2 The hardware I used

For persistent storage I used a W25Q64 NOR flash (64Mbit, 8MByte). The microphone is an ordinary electret condenser capsule. Its signal is amplified by an MCP6022 operational amplifier and digitised by an MCP3201 12-bit ADC, which the FPGA reads over SPI. The speaker is an ordinary 8Ω speaker driven from the PWM output through a low side MOSFET. Power comes from a single 12V supply that a DFRobot DFR1202 buck module steps down to 3.3V for the digital part, while the 12V rail directly feeds the power LED and the speaker stage. Several RLZ44N MOSFETs do the low side switching, plus the usual capacitors and diodes.

The slave device, the one that receives orders over the air from this master, is much simpler and is documented elsewhere: it is an ESP32 wired at register level to an I2S microphone on one side and, through I2S again, to a class D amplifier and a speaker on the other.

2 System overview

The system is a two node star. The master is the hub, the slave is the single spoke, and a web dashboard opened in any browser is an optional viewer. Figure 1 shows the picture.

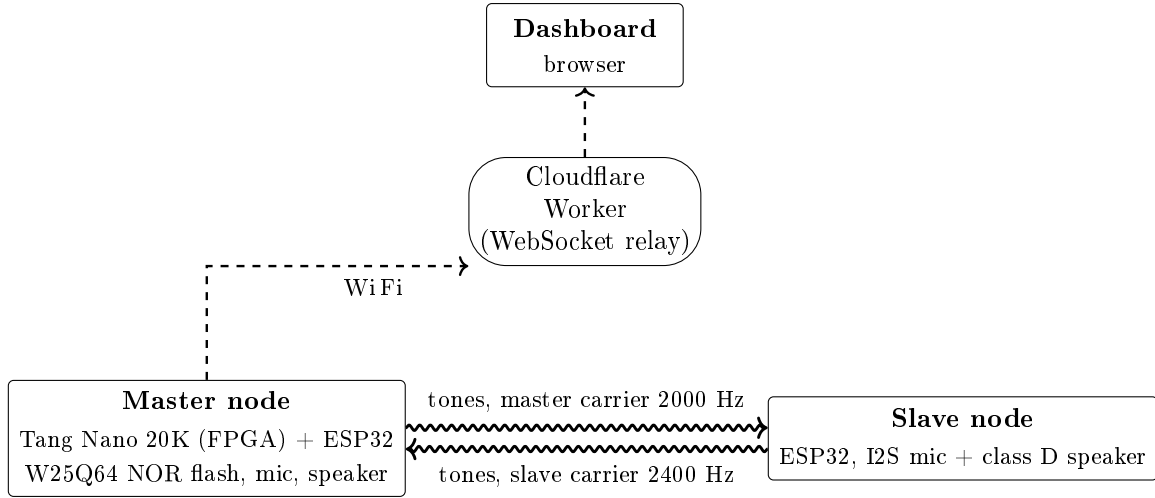


Figure 1: System level view. The only link between master and slave is acoustic (wavy arrows). The WiFi link only carries dashboard traffic; the slave does not need it.

The acoustic link is the only path between master and slave. There is no shared cable, no Bluetooth, no I2C. Everything the two say to each other, presence requests, identification, acknowledgements, abort, has to be encoded into tones and demodulated on the other side (Section 12). The WiFi link is only there so I can watch and command the system from a browser; it is not part of the acoustic protocol.

Inside the master, the ESP32 and the FPGA split the work cleanly. The FPGA does everything that is timing critical or sample rate bound: it reads the ADC, computes the FFT, decodes the tones, generates the output PWM tone, and stores the log on the NOR flash. The ESP32 does everything that is decision bound: it runs the protocol state machine, talks to the dashboard, and tells the FPGA which tones to emit. The two are connected by two UARTs, one carrying the decoded and to be emitted acoustic symbols, the other carrying the flash storage commands.

3 The FPGA SoC

The Gowin GW2AR-18 holds the custom SoC. It is a 20k LUT class FPGA, and the AR variant has an 8 MByte SDR SDRAM die inside the same package, reachable on dedicated fabric pins (no external SDRAM soldering). Figure 2 is the top level block diagram.

3.1 Block diagram

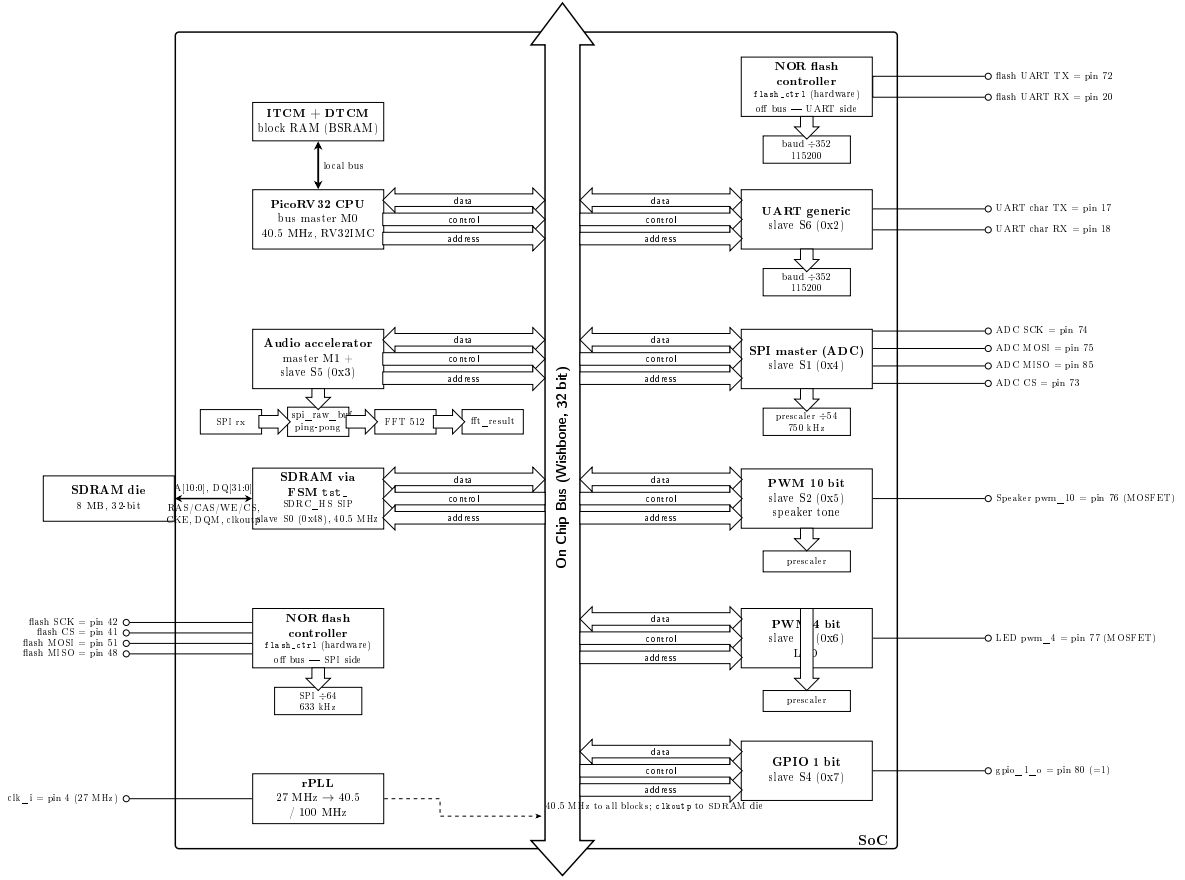


Figure 2: SoC top level (black and white). Each block reaches the central 32 bit Wishbone bus with three thick arrows: *data* is bidirectional, while *control* and *address* arrows point toward the bus for the two masters (PicoRV32, audio accelerator) and toward the block for the slaves, indicating signal direction. The audio accelerator is both a master (M1, to move samples) and a slave (S5, for its control registers). ITCM and DTCM are CPU local block RAM, not on the Wishbone. The character UART generic slave S6 (0x2) reaches the ESP32 on pins 17 and 18 and carries the protocol characters (boot banner, decoder output, soft core). The **NOR flash controller** (*flash_ctrl1*) is a self contained hardware block, *not* on the Wishbone bus and with no CPU in the loop: its UART side talks to the ESP32 on pins 72 and 20 (storage commands), and its SPI side drives the W25Q chip on pins 42, 41, 51, 48. The two *flash_ctrl1* boxes in the figure (UART side, top right; SPI side, lower left) are the *same* block, drawn at its two pin groups; neither has bus arrows, because it does not touch the bus. Every external connection carries its real FPGA pin number from the constraints file.

3.2 The on chip bus and the address map

The interconnect (*wb_interconnect.vhd*) is a shared bus crossbar with **two masters and seven slaves**. The two masters are the PicoRV32 CPU (port M0) and the audio accelerator (port M1). When both request the bus in the same cycle, M1 wins: the accelerator is sample rate bound and must not be stalled, while the CPU can wait a cycle. The slave that answers a transaction is chosen by decoding the top five address bits *addr*[31:27].

Five bits, not four, because of where the soft core keeps its own memory. The IP treats every address with *addr*[31:29]=000 (0x0-0x1FFF_FFFF) as *internal* (its ITCM, DTCM and native UART): an external read there never completes, it just stalls the core. The SDRAM slave therefore lives at 0x4800_0000 (*addr*[31:29]=010) and not at 0x1000_0000; since that base falls

on a half nibble, with bit 27 set, the decoder needs the fifth bit to tell it apart from the ADC SPI at 0x4000_0000. The NOR flash is **not on this bus at all**: it is a self contained hardware block (`flash_ctrl`) that receives storage commands from the ESP32 on its own UART (pins 72 and 20) and drives the W25Q over its own SPI (pins 42, 41, 51, 48), with no CPU in the loop, exactly as it worked before the soft core was added. Table 1 is the real map from the RTL.

Table 1: Address map, decoded from `addr[31:27]` (five bits). The NOR flash is *not* on this bus: it is an independent hardware block (`flash_ctrl`), so the CPU map has seven slaves. SDRAM sits at 0x4800_0000, not in the 0x0–0x1 range, because that range is the soft core’s own internal space (see text).

Base	addr[31:27]	Slave	Block
0x4800_0000	01001	S0	SDRAM (through the FSM <code>tst_</code>)
0x2000_0000	00100	S6	UART generic (characters, pins 17/18)
0x3000_0000	00110	S5	Audio accelerator control registers
0x4000_0000	01000	S1	SPI master (reads the ADC)
0x5000_0000	01010	S2	PWM 10 bit (speaker tone)
0x6000_0000	01100	S3	PWM 4 bit (LED)
0x7000_0000	01110	S4	GPIO 1 bit

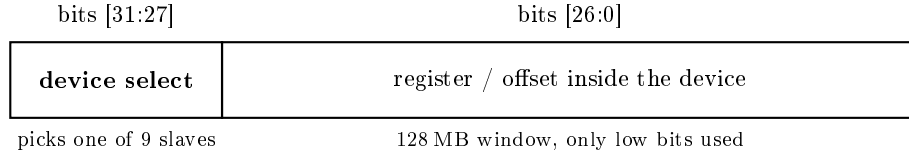


Figure 3: The 32 bit address is split in two. The top five bits `addr[31:27]` pick the slave (Table 1); the low 27 bits are the offset inside that slave, of which each peripheral decodes only the few it needs (the per device register maps below).

A transaction works the standard Wishbone way. The active master drives `adr`, `dat`, `we`, `sel` and asserts `cyc` and `stb`. The interconnect broadcasts the address and write data to all slaves but only routes `stb` and `cyc` to the one slave selected by `addr[31:27]`; the read data and the `ack` are multiplexed back from that same slave. A peripheral therefore needs to do nothing more than raise `ack` for one cycle when it is addressed. The decode is purely combinational on five bits, and any address that matches none of the nine windows falls to a sink that returns zero with no `ack`.

3.3 Arbitration

There are two masters on the bus, so it needs an arbiter. I used static priority, not round robin: M1 (the audio accelerator) always wins over M0 (the CPU). In the RTL this is one line, the select signal `sel_m1 = m1_cyc AND m1_stb`; when it is high the mux puts M1’s `adr/dat/we/sel/cyc/stb` on the bus, otherwise M0’s. The `ack` is steered the same way: `m0_ack = slv_ack AND NOT sel_m1` and `m1_ack = slv_ack AND sel_m1`. So while M1 holds the bus, M0 simply receives no `ack` and stalls on its current access until M1 releases, which is the standard Wishbone way to make a master wait, with no extra logic.

The reason for the fixed M1 over M0 priority is timing. The accelerator is sample rate bound: it must move each batch of FFT results out before the next audio window is ready, and it cannot be made to wait. The CPU has no hard deadline, so it is the one that yields. The cost to the CPU is small in practice, because M1 only takes the bus in short bursts: after each audio window the accelerator drains 512 result words to SDRAM, which is a few tens of microseconds once

every 10.9 ms (Section 6), so the CPU is blocked for well under 1 % of the time and never for long enough to matter.

3.4 The simple slaves: UART and GPIO register maps

The two slaves without a section of their own are the generic UART (S6) and the GPIO (S4). The UART is the widest register map in the design; the GPIO is a single bit.

Table 2: UART generic registers (base 0x2000_0000) and GPIO (base 0x7000_0000).

Device	Offset	Dir	Function
UART	0x00	W	transmit a byte (<code>dat[7:0]</code>)
UART	0x00	R	<code>bit[8]=rx_valid</code> , <code>bit[7:0]=received byte</code> (read clears valid)
UART	0x04	W	enable
UART	0x08	W	disable
UART	0x0C	W	<code>baud_div = clk / baud</code>
UART	0x10	W	format: <code>dat[1:0]=parity</code> , <code>dat[2]=stop bits</code> , <code>dat[6:3]=data bits-5</code>
UART	0x14	R	<code>bit[1]=rx_valid</code> , <code>bit[0]=tx_busy</code>
GPIO	0x00	W	<code>dat[0]</code> drives the output pin

4 Clock tree

A single 27 MHz clock enters the FPGA on pin 4 (`clk_i`). The active rPLL (`gowin_rpll.vhd`) is configured `IDIV_SEL=1`, `FBDIV_SEL=2`, so it divides by 2 and multiplies by 3:

$$f_{\text{clkout}} = 27 \text{ MHz} \times \frac{3}{2} = 40.5 \text{ MHz}.$$

The second output `clkoutp` is the same 40.5 MHz but phase shifted; with `PSDA_SEL=1` and `ODIV_SEL=16` the shift is one of 16 steps, $360^\circ/16 = 22.5^\circ$, which at 40.5 MHz (period 24.7 ns) is $24.7 \text{ ns} \times 22.5/360 = 1.54 \text{ ns}$.

There are three clocks, and which block runs on which matters. The 27 MHz (`clk_i`) is the PLL reference and also directly clocks the PWM speaker, the PWM LED, the GPIO and the boot banner. The 40.5 MHz (`clkout`) is the main clock: CPU, Wishbone interconnect, SPI for the ADC, audio accelerator, SDRAM controller, flash SPI, UART and the tone decoder. The 40.5 MHz phase shifted (`clkoutp`) clocks only the SDRAM die, to centre its sampling window (Section 5). Table 3 gives the base clock and the divider each block uses to reach its I/O rate.

Table 3: Base clock and internal divider per block.

Block	Base clk	Divider	Result
SPI for NOR flash	40.5 MHz	$\div 64$	633 kHz SCK, 1 bit = $1.58 \mu\text{s}$ (slow on purpose, Sec. 10)
SPI for ADC	40.5 MHz	$\div 54$	750 kHz SCK; 16 SCK = 864 cyc/sample $\rightarrow$ 46.875 kHz
UART	40.5 MHz	$\div 352$	115,057 baud (115200 within 0.1 %)
Audio accelerator	40.5 MHz	paced by ADC	one sample per conversion, 46.875 kHz
SDRAM controller	40.5 MHz	none	die clocked by <code>clkoutp</code> (+1.54 ns)
PWM speaker / LED	27 MHz	DDS phase acc	two tone sine, carrier $27\text{M}/256 = 105.5 \text{ kHz}$
GPIO	27 MHz	none	static output

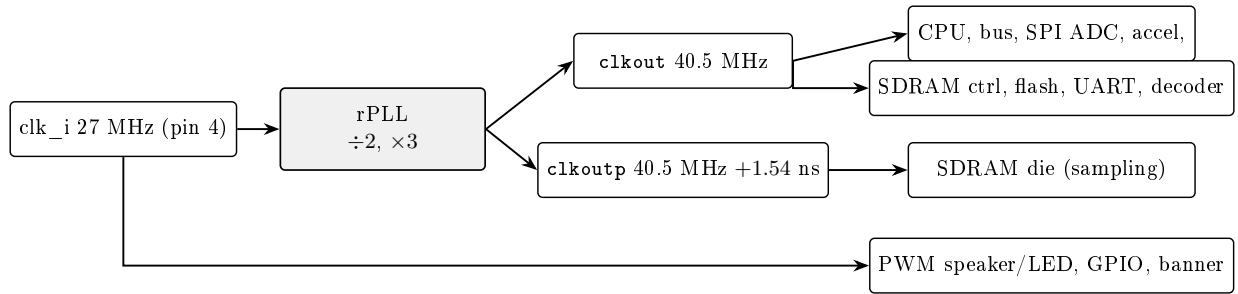


Figure 4: Clock tree. The 27MHz input is the PLL reference and also clocks the PWM, GPIO and boot banner directly. The rPLL produces 40.5 MHz (**clkout**) for the CPU, bus and most peripherals, and a +1.54 ns phase shifted copy (**clkoutp**) that clocks only the SDRAM die.

5 SDRAM controller

The 8 MByte die is internal to the GW2AR-18 package and speaks JEDEC SDR SDRAM. It is not driven directly. A **memory_arbiter** sits in front of it and drives the Gowin SDRC_HS controller IP. The arbiter has two ports, which is the double input shown on the SDRAM block in Figure 2: port M0 is the CPU read path (reached as bus slave S0), port M1 is the audio accelerator write path, wired direct and not through the main crossbar. M1 has priority over M0, because the accelerator must drain its FFT results without waiting while the CPU can.

The arbiter is a small FSM around the IP. A write is M1_CMD (hold **cmd_en** until **cmd_ack**) → M1_WR_WAIT (14 cycles to cover WRITE + t_{WR} + PRECHARGE + t_{RP}) → M1_ACK. A read is M0_CMD → M0_RD_WAIT (5 cycles, CAS latency 3 plus margin, data valid on **0_sdrc_data**) → M0_ACK. The controller side uses these die pins: **0_sdram_dq[31:0]** (32 bit data), **0_sdram_addr[10:0]**, **0_sdram_ba[1:0]** (4 banks), **0_sdram_dqm[3:0]**, plus CKE / CS / RAS / CAS / WE and the clock. The whole path runs on **clk_sdram** = 40.5MHz; the die is clocked by **clkoutp**, the +1.54ns phase shifted copy.

The Mode Register is programmed with these values:

- CAS latency = 3 (the read data appears 3 clocks after the READ command);
- burst length = 1 (I read and write one 16 bit word at a time, because the accelerator and the CPU access scattered addresses, not long bursts);
- burst type = sequential;
- write burst = single location.

I did not use the longer burst lengths or the full page burst mode. I spent a fair amount of time looking at full page / fast page access, because on paper it gives the highest throughput, but it only pays off when you stream long contiguous blocks. My traffic is the opposite: the accelerator writes one FFT bin here, the CPU reads one word there. With burst length 1 every access is self contained, the controller stays simple, and I never pay for opening a row I am not going to reuse. So the design choice was burst length 1 on purpose, not by omission.

5.1 Bring up sequence

The order below was found by testing, not by copying a figure. Each step is the problem I saw and the parameter that fixed it:

1. READ issued right after power up → garbage on every access. The die is not ready yet.
Fix: assert **CKE**, wait $\geq 200 \mu s$ for the internal regulator.

2. write then immediate read back → still wrong on the first few accesses, then correct. The refresh counter was not locked. Fix: two AUTO REFRESH commands before programming the Mode Register.
3. PRECHARGE ALL before the refreshes → needed to put all four banks in the idle state first.

So the bring up is: CKE high and wait $\geq 200\mu\text{s}$; PRECHARGE ALL; two AUTO REFRESH; program the Mode Register (CL= 3, BL= 1, sequential, single write); after that, ACTIVATE / READ / WRITE / PRECHARGE are allowed.

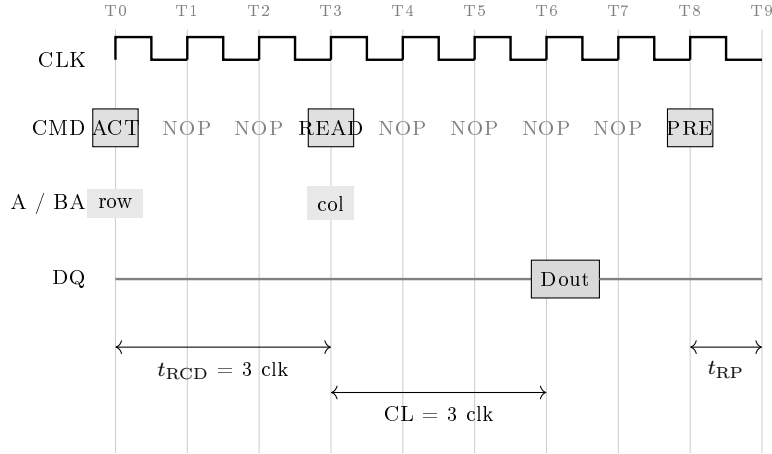


Figure 5: SDRAM single word read at 40.5 MHz (one clock = 24.7 ns). ACTIVATE at T0 opens the row, t_{RCD} later the READ at T3 selects the column, and with CAS latency 3 the word is valid on DQ at T6 ($3 \times 24.7 = 74 \text{ ns}$ after the READ). PRECHARGE at T8 closes the row, after which t_{RP} must elapse before the next ACTIVATE. With burst length 1, exactly one word comes out per READ.

Two more numbers came from the bench. First, skew: at 40.5 MHz the clock period is 24.7 ns, and I was reading 0xFFFE for 0xFFFF on a small fraction of accesses, because the latching edge landed at the edge of the data valid window instead of its centre. Driving the die from `clkoutp`, the phase shifted copy, fixed it: with `PSDA_SEL= 1` and `ODIV_SEL= 16` the shift is $360^\circ/16 = 22.5^\circ = 1.54 \text{ ns}$, enough to move the sampling edge into the centre of the window, and the bit errors disappeared. Second, refresh: all 8192 rows must be refreshed every 64 ms, so one AUTO REFRESH every $64 \text{ ms}/8192 = 7.81 \mu\text{s}$, which is $7.81 \mu\text{s} \times 40.5 \text{ MHz} = 316$ cycles. My first controller refreshed too rarely and rows lost charge after a few seconds; the 7.81 μs counter made retention solid.

6 Audio accelerator and tone decoder

The receive path is built from two state machines that run in parallel and a ping-pong buffer between them. The accelerator turns microphone samples into a spectrum in SDRAM and raises an IRQ; the CPU starts it once and then, on each IRQ, reads that spectrum back over the bus (slave S0) and decodes the tones *in software*, consistent with the design goal of keeping the high level logic on the soft core. It never touches a raw sample. The capture FSM (`dma_proc`) reads the ADC over SPI and writes samples into `spi_raw_buf`, a 1024 word block RAM holding two 512 sample windows. The FFT FSM (`fft_proc`) takes the window just filled, feeds it to the Gowin FFT core, collects the 512 spectrum words in `fft_result_buf`, drains them to SDRAM and raises the IRQ. While one half of `spi_raw_buf` is being processed, the other half keeps filling, so no sample is lost.

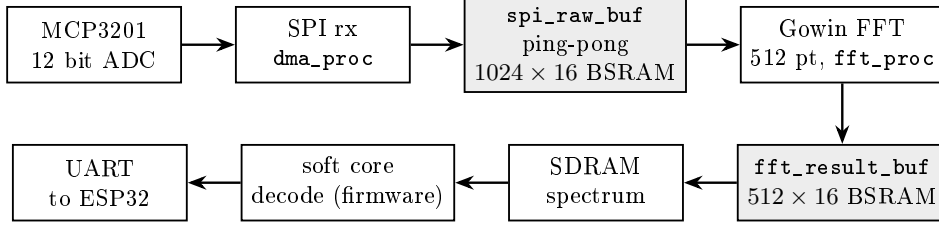


Figure 6: Receive pipeline (black and white). The capture FSM fills one half of the ping-pong block RAM while the FFT FSM processes the other half into a spectrum in SDRAM; the soft core then reads the spectrum back over the bus (S0) and, in firmware, decodes one character per tone and sends it to the ESP32.

6.1 Timing in cycles and in seconds

Everything is paced by the ADC. One SCK period is the prescaler, 54 bus cycles, so $54/40.5 \text{ MHz} = 1.333 \mu\text{s}$ (SCK = 750 kHz). One MCP3201 conversion is 16 SCK periods, so

$$t_{\text{sample}} = 16 \times 54 = 864 \text{ cycles} = \frac{864}{40.5 \text{ MHz}} = 21.33 \mu\text{s}, \quad f_s = \frac{1}{21.33 \mu\text{s}} = 46.875 \text{ kHz}.$$

One window is 512 samples, so the time to fill it (and the time between IRQs in steady state) is

$$t_{\text{window}} = 16 \times 512 \times 54 = 442,368 \text{ cycles} = \frac{442,368}{40.5 \text{ MHz}} = 10.92 \text{ ms} \Rightarrow \text{IRQ rate} = 91.6 \text{ Hz}.$$

The processing of one window must fit inside that 10.92 ms. Feeding 512 samples to the FFT, the FFT pipeline, and collecting 512 outputs cost on the order of $3 \times 512 \approx 1500$ cycles at 40.5 MHz, i.e. about $37 \mu\text{s}$. Draining the 512 result words to SDRAM, at roughly 10 cycles of the 40.5 MHz clock per word, is $512 \times 10/40.5 \text{ MHz} = 126 \mu\text{s}$. So

$$t_{\text{process}} \approx 37 + 126 = 163 \mu\text{s} \ll t_{\text{window}} = 10.92 \text{ ms} \quad (\text{ratio} \approx 67).$$

This is the condition that makes the pipeline work: $t_{\text{process}} \ll t_{\text{window}}$, so processing one window ($163 \mu\text{s}$) finishes long before the next window is full (10.92 ms). The ping-pong buffer is what lets the two overlap, and the margin is large ($67\times$), so a sample is never at risk of being overwritten before it is read.

6.2 The two state machines

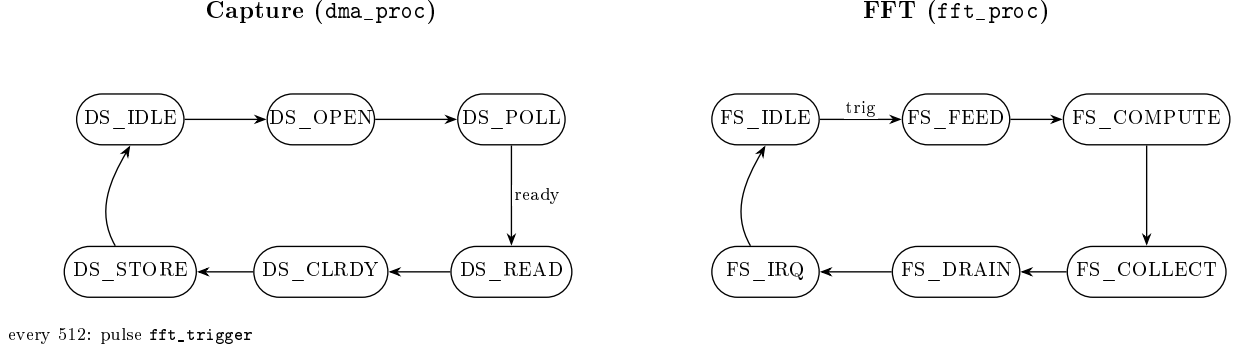


Figure 7: The two parallel FSMs. Capture reads one ADC sample per pass (`DS_OPEN` starts the SPI, `DS_POLL` waits for the conversion, `DS_READ/DS_STORE` write it into the ping-pong buffer) and pulses `fft_trigger` every 512 samples. The FFT FSM, on that trigger, feeds the window (`FS_FEED`), runs the FFT (`FS_COMPUTE`), collects the bins (`FS_COLLECT`), writes them to SDRAM (`FS_DRAIN`) and raises the IRQ (`FS_IRQ`). Inside the accelerator the capture path has priority on the M1 bus port, so if both want the bus during `FS_DRAIN` the FFT simply stalls (it gets no ack) and resumes when capture releases.

Table 4: Audio accelerator control registers, base `0x3000_0000`.

Offset	Dir	Function
0x04	W	start continuous capture + FFT
0x08	W	stop
0x0C	W	set SDRAM output base: <code>dat[20:0]</code> = 21 bit word address

6.3 FFT size and the tone bins

The FFT is 512 point and runs on samples taken at 46.875 kHz, so the bin spacing is

$$\Delta f = \frac{46875 \text{ Hz}}{512} = 91.6 \text{ Hz}.$$

A tone at frequency f lands in bin $k = \text{round}(f/\Delta f)$. The five tones the protocol uses, and the bins the decoder watches, are in Table 5.

Table 5: Protocol tones and their FFT bins ($\Delta f = 91.6 \text{ Hz}$, $N = 512$).

Tone	Frequency	Bin k
master carrier	2000 Hz	22
slave carrier	2400 Hz	27
EOF marker	2900 Hz	32
data bit 0	3500 Hz	39
data bit 1	4500 Hz	50

The smallest gap between two used bins is between the EOF at bin 32 and bit 0 at bin 39, that is 7 bins or 641 Hz. That gap matters for the decoder, as explained next.

6.4 The tone decoder, with its checks

For each of the five candidate bins, the decoder decides whether that tone is present. It applies, in order: a local maximum test, the candidate bin magnitude must be greater than or equal to its six neighbours within ± 3 bins ($k-3 \dots k+3$); an amplitude test, the magnitude must exceed a threshold derived from the running noise floor; and a parabolic interpolation on the three points $k-1, k, k+1$ to confirm the peak is genuinely centred on k and not the skirt of a neighbour. If all three pass, the decoder emits the character for that tone on the UART.

The ± 3 neighbourhood, rather than a strict ± 1 , is used because the Hann window applied before the FFT spreads a pure tone over two or three bins.

7 SPI master for the ADC, and the microphone front end

The receive chain in front of the FPGA is analog, and getting it described correctly matters. The electret microphone produces a few millivolts riding on no particular DC level. An MCP6022 operational amplifier, wired as a non inverting stage, does two jobs: it biases the signal to mid rail and it amplifies it. The bias comes from a $47\text{k}\Omega / 47\text{k}\Omega$ divider from the 3.3 V rail, which sets the operating point at $3.3/2 = 1.65\text{ V}$ so that the amplified waveform swings symmetrically inside the ADC input range. After the amplifier an RC low pass acts as an anti aliasing filter before the ADC. The MCP3201 is a 12 bit SPI ADC: it is the SPI slave, and the FPGA SPI master clocks it to read each conversion. So the chain is

electret mic $\rightarrow$ MCP6022 (gain + 1.65 V bias) $\rightarrow$ RC anti alias
 $\rightarrow$ MCP3201 (12 bit SPI ADC) $\rightarrow$ FPGA SPI master.

Table 6: SPI master (ADC) register map, base 0x4000_0000 (offset in the low bits).

Offset	Dir	Function
0x00	R	12 bit ADC sample; the read acks only when <code>data_ready=1</code>
0x04	W	start a conversion
0x08	W	stop
0x0C	W	clear the <code>data_ready</code> flag

7.1 Why a custom SPI master and not a generic one

The MCP3201 has a fixed and slightly awkward frame: after the chip select goes low there is a sample/null period, then the 12 data bits come out most significant first. I wrote a dedicated SPI master for it rather than reuse a generic byte oriented engine, because here I do not send anything (MOSI is unused), I only need to clock 16 SCK periods, throw away the leading null bit, and capture exactly 12 bits into a register, then raise a data ready flag the accelerator polls. Tying the frame to the part lets the sample rate be exact and constant, which the FFT needs.

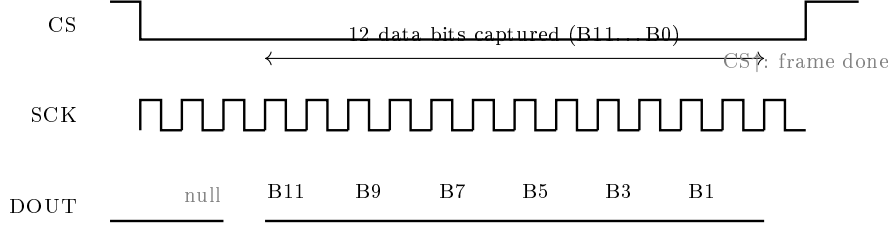


Figure 8: MCP3201 read as seen by the SPI master. CS goes low, the master issues 16 SCK periods at 750 kHz, discards the leading null bit, captures the 12 data bits B11 down to B0 most significant first, then raises CS to end the frame. One frame is $16/750 \text{ kHz} = 21.3 \mu\text{s}$, which sets the 46.9 kHz sample rate.

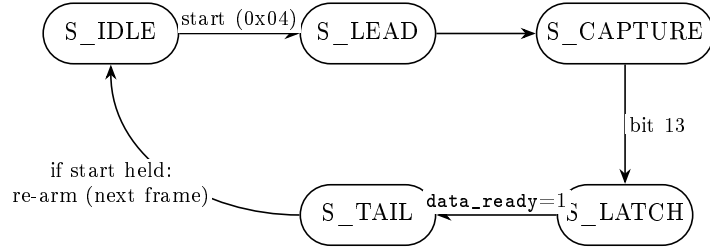


Figure 9: SPI master (ADC) frame FSM, from `spi_master.vhd`. A write to 0x04 asserts CS low and starts one MCP3201 frame of 16 SCK periods at 750 kHz (PRESCALER=54 from the 40.5 MHz clock). S_LEAD clocks the leading null bit, S_CAPTURE shifts MISO in most significant first for the 12 data bits, S_LATCH writes them into the 0x00 register and sets `data_ready`, S_TAIL raises CS to close the $21.3 \mu\text{s}$ frame. While 0x04 (start) stays asserted the frame re-arms, which gives the continuous 46.9 kHz sample stream the FFT needs; a write to 0x0C clears `data_ready`.

8 PWM speaker and LED

Table 7: PWM register map (speaker base 0x5000_0000, LED base 0x6000_0000). F_WIDTH is the period field width.

Offset	Dir	Function
0x04	W	load two tones and start: $\text{dat}[\text{F_WIDTH}-1:0] = f_1$, $\text{dat}[2\text{F_WIDTH}-1:\text{F_WIDTH}] = f_2$
0x08	W	stop

The transmit side is not a square wave generator, even though that would be the simplest thing to do. A square wave at frequency f carries all its odd harmonics ($3f, 5f, 7f, \dots$) at large amplitude; on the receiver those harmonics land in other FFT bins and can be read as tones that were never sent, and they waste output energy that should sit in the wanted bin. So the block (`pwm_generic_master`) synthesises the tone with direct digital synthesis (DDS) and a sine lookup table, which keeps almost all of the energy at f .

The numbers, all at the 27 MHz PWM clock. A write to 0x04 loads two 16 bit frequencies f_1 and f_2 . Each one drives a 24 bit phase accumulator that adds a phase increment every clock,

$$\Delta\phi = \text{round}\left(\frac{f \cdot 2^{24}}{27 \text{ MHz}}\right) = (f \cdot K + 2^{23}) \gg 24, \quad K = \text{round}\left(\frac{2^{48}}{27 \text{ MHz}}\right) = 10,424,999,$$

where the right hand form avoids a hardware divide. The top 8 bits of each accumulator index a 256 entry, 8 bit sine table whose entry i is $\text{round}(128 + 127 \sin(2\pi i/256))$, centred at 128. The

two sine samples are added and halved into one 8 bit duty value, so a single pin carries both tones at once, which is exactly what the protocol needs since it sends superimposed tone pairs.

That 8 bit duty feeds a free running 8 bit PWM comparator, so the carrier is $27\text{ MHz}/2^8 = 105.5\text{ kHz}$, far above the audio band. The RC low pass on the MOSFET gate then recovers the analog two tone sine and rejects the carrier.

The amplitude is not switched on and off hard either. A hard on/off edge is a step, which is broadband and would add a click plus spectral splatter into the neighbouring bins. Instead an attack/release envelope ramps the AC part of the duty (the swing around the 128 centre) from 0 to full and back over $\text{RAMP_MS} = 4\text{ ms}$, one amplitude step every $\text{RAMP_TICK} = (27\text{ MHz}/1000 \cdot 4)/255 \approx 423$ clocks. The phase accumulators keep running through the release, so the tone fades out instead of cutting. The envelope is the small FSM in Figure 10.

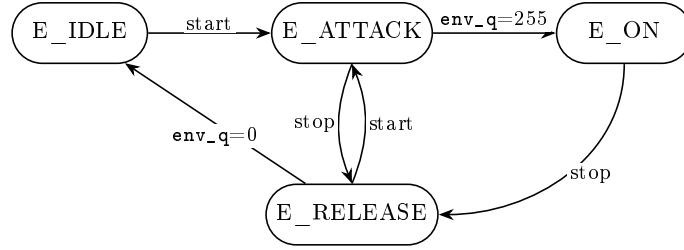


Figure 10: Attack/release envelope FSM in the PWM block. **start** is set by the `0x04` write and cleared by `0x08`. Attack ramps the amplitude up, **E_ON** holds it, release ramps it down; a stop during attack or a start during release switches directly between the two ramps, so a retriggered tone never clicks.

The duty stream drives the gate of a low side MOSFET (an RLZ44N); the speaker sits between the 12 V rail and the MOSFET drain, with a flyback diode across it to absorb the inductive kick when the MOSFET switches off. There is no operational amplifier in the speaker path (the op amp belongs to the microphone input chain, Section 7): the chain is just PWM, RC filter, MOSFET, speaker.

The LED indicator is the 4 bit PWM peripheral, used the same way: its output drives the gate of a second low side MOSFET that switches a 6 W power LED on the 12 V rail, with a series resistor $R = (V_{\text{in}} - V_f)/I_{\text{LED}}$ setting the current. The 4 bits give 16 brightness levels.

A tone is emitted for a fixed number of cycles, then the transmit path leaves a short silence before the next tone, so the receiver has time to finish processing one symbol before the next arrives.

9 NOR flash controller

The flash controller is the largest block by line count and the one that caused the most trouble on the bench. It takes high level requests from the ESP32 over a dedicated UART (append a 16 byte record, save the 32 byte settings, read a slot, clear the log) and turns each one into the right sequence of SPI commands to the external W25Q64 chip.

9.1 Memory layout

I use the first 12 KB of the chip, three 4 KB sectors (4 KB is the natural erase granularity of the part). Sector 0 at `0x0000` holds the 32 byte settings (a `0xA5` magic byte, a 15 byte device name, a 2 byte auto presence interval, a 1 byte retry count, and padding). Sector 1 at `0x1000` and sector 2 at `0x2000` hold the log, 256 slots of 16 bytes each, so 512 slots together, managed as a ring buffer. Each 16 byte record is: 2 bytes sequence number, 1 byte type character (B boot, R registration, P presence, N note, E event), 4 bytes timestamp, 1 byte value, 8 bytes ASCII text. Figure 11 draws this map.

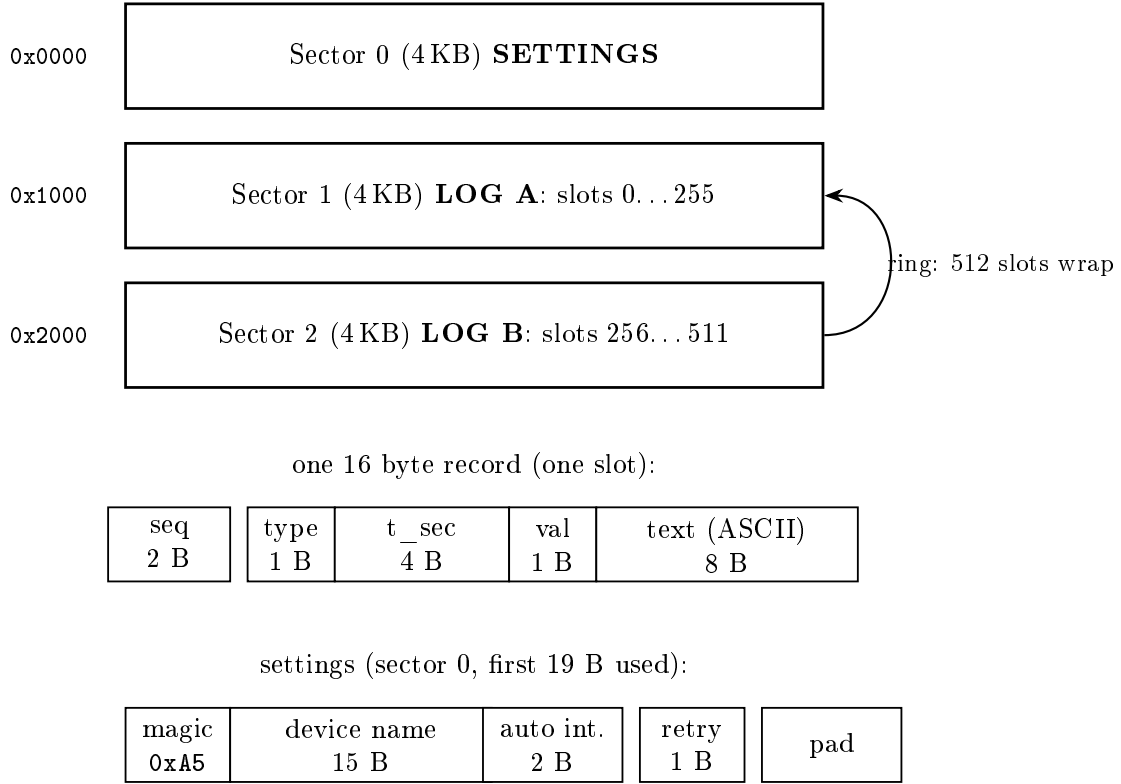


Figure 11: Flash memory map (low 12 KB). Three 4 KB sectors: sector 0 holds the 32 byte settings, sectors 1 and 2 hold the 512 slot log ring buffer. Below, the 16 byte record and the settings sector are cut into their byte fields.

9.2 State machine

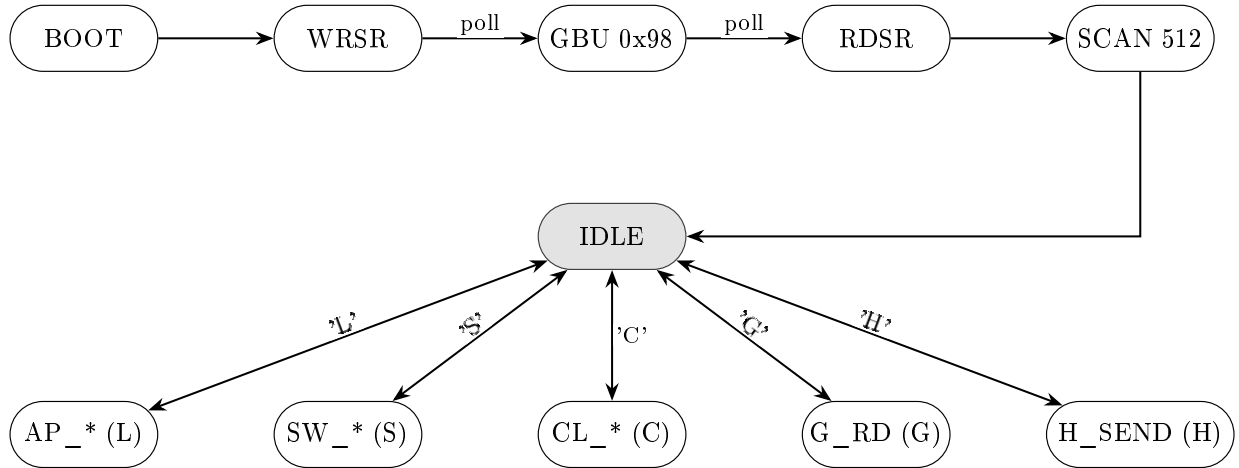


Figure 12: flash_ctrl state machine. At power up it unlocks the chip (WRSR clears the protection bits BP0..BP2 and SRP and sets QE, then a Global Block Unlock), reads the status register back to confirm, and scans all 512 slots to find the head of the ring buffer, then sits in IDLE. Each UART opcode (L append, S save settings, C clear, G read, H read head) is entered from IDLE on its character and returns to IDLE when its SPI sequence finishes; the double headed arrows mean exactly that go and return.

Append (opcode L) as an SPI sequence. An append is not a single command but a short ordered sequence of SPI frames to the chip, with a status poll between the slow ones. Figure 13 is that sequence in time order.

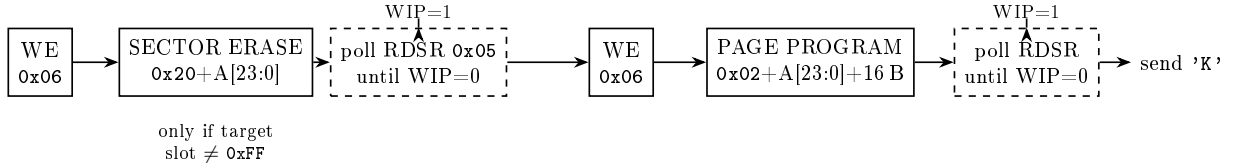


Figure 13: Append (opcode L) as the time ordered sequence of SPI frames the controller sends to the W25Q. Each solid box is one CS low frame; the dashed boxes are status polls (read `0x05`, loop while the WIP write in progress bit is 1). The controller first probes the target slot; the erase happens only if that slot is not `0xFF`, because a NOR program can only turn 1 bits into 0, so writing over a used slot would corrupt it. A read (opcode G) is instead a single frame: `0x03` + 24 bit address, then clock out $N \times 16$ data bytes.

9.3 SCAN: finding the head after power up

At power up the controller does not know where the last valid record is. It reads byte 2 (the type byte) of every one of the 512 slots and treats a slot as valid only if that byte is one of B, R, P, N, E. Anything else (the `0xFF` of an erased slot, or garbage from an aborted write) counts as empty. The head becomes (highest valid slot + 1) mod 512. This is more robust than stopping at the first `0xFF`, which I tried first: that version stopped at any hole, even when valid records sat past the hole, which is exactly what a failed retry leaves behind.

9.4 Unlocking the chip and the polling timeout

Two settings in this block came directly from bench failures, both documented with their numbers in Section 10. At boot the controller issues a write enable, a WRSR that writes `0x00` to status register 1 (clearing the block protect bits BP0..BP2 and the SRP bit) and `0x02` to status register 2 (setting QE, which on this part also disables the /HOLD# and /WP# pin functions), then a Global Block Unlock (`0x98`). The polling timeout that waits for WIP to clear is set to 25 million bus cycles, about 620 ms, because a sector erase on this part can take up to 400 ms and the first version timed out at 5 ms and silently abandoned every erase.

9.5 Generic SPI engine register map

The reworked storage SPI is the generic, register driven engine. Its registers are:

Table 8: Generic SPI engine register map.

Offset	Dir	Function
0x00	R	RXDATA, the last received byte
0x04	W	TXDATA, write a byte and start a full duplex transfer
0x08	W	CTRL, bit 0 = chip select (1 asserts CS low, held across bytes)
0x0C	R	STATUS, bit 0 = busy, bit 1 = done

9.6 Note on the current architecture

While reworking the SoC I moved the storage SPI onto the bus as a generic, register driven SPI engine (the SPI NOR block in Figure 2), so that the W25Q sequence above can be driven by the CPU in firmware rather than by a dedicated hardware state machine. The sequence itself, the unlock, the probe, the erase before program, the polling, is unchanged; what changed is who issues it.

10 Signal integrity and decoupling

None of the problems in this section showed up in simulation. All of them showed up on the bench, and the fixes are a mix of a few capacitors, a slower clock, and firmware retries. The common thread is that the W25Q64 sits on a breakout board connected to the FPGA by about 10 cm of Dupont jumper wires, which is a poor environment for both power delivery and signal edges.

10.1 Brownout during sector erase

To erase a 4 KB sector the chip runs an internal charge pump that draws around 80 mA in peaks for about 30 ms (typical), up to 400 ms worst case. To program one record the draw is lighter, around 25 mA for under 3 ms. The supply reaches the chip through the jumper wires, so any current spike shows up as a voltage droop at the chip.

The droop over a decoupling capacitor follows $\Delta V = It/C$. With no capacitor at all (only the tens of pF of the wire), the 80 mA erase current collapsed the 3.3 V rail almost instantly. The chip browned out mid erase, lost its write enable, silently rejected the program that followed, and the firmware saw a success that had not happened.

Adding a 1 μ F ceramic across VCC and GND at the chip gives

$$\Delta V = \frac{0.080 \times 0.030}{1 \times 10^{-6}} = 2.4 \text{ V},$$

which still drops the rail to 0.9 V, below the 2.7 V minimum, so log writes started working but settings saves (erase plus program) still failed. Adding a second capacitor of 10 μ F in parallel, for 11 μ F total, gives

$$\Delta V = \frac{0.080 \times 0.030}{11 \times 10^{-6}} = 0.22 \text{ V},$$

so the rail only drops to 3.08 V, safely above 2.7 V, and the brownout is gone. The two values cover different bands: the 1 μ F has low ESR/ESL and handles the fast SPI edges, the 10 μ F holds up the slow charge pump current. The minimum capacitance to stay inside a 0.6 V budget is $C \geq It/\Delta V = 0.080 \times 0.030/0.6 = 4 \mu\text{F}$, so 11 μ F has comfortable margin.

10.2 SPI clock too fast for the wiring

Separately, the first SPI clock of 5 MHz was too fast for 10 cm of jumper with a single ground return. Long writes came out truncated: a 16 byte record would arrive with byte 10 onward stuck at 0xFF. I lowered the clock in steps and watched the first attempt success rate: 5 MHz was bad, 2.5 MHz better, 1.27 MHz mostly good, and 633 kHz (divide by 64) almost always good. At 633 kHz a bit lasts 1.58 μ s, long enough that the ringing on the wire settles well before the chip samples.

10.3 Retry and verify in firmware

Even at 633 kHz an occasional write still fails (roughly 1 in 100 on settings, rarer on log records). The firmware does up to 5 attempts: send, wait for the acknowledgement, read the slot back, compare all 16 bytes, retry if they differ. A failed attempt burns one ring buffer slot, but the SCAN at boot ignores those because their type byte is not in the valid set, so nothing is lost permanently. Table 9 is the measured reliability after all three fixes.

Table 9: Measured first attempt reliability after 633 kHz SPI, 11 μ F decoupling and firmware retry/verify.

Operation	First attempt	After up to 5 retries
log record write (16 B)	$\approx 95\%$	$\approx 100\%$
settings save (32 B, with erase)	$\approx 85\%$	$\approx 100\%$
read back of a record	100%	100%
JEDEC ID read at boot	100%	100%

A separate hardware lesson, worth recording: an indicator LED that I had wired with its anode straight to an FPGA pin and its cathode to the common ground, with no series resistor, was injecting tens of mA into the ground return. That current, through the impedance of the shared ground wire, moved the ground reference enough to corrupt the microphone ADC readings (the FFT then "heard" tones that were not there) and to disturb the UART and SPI at the same time. Removing the resistorless LEDs fixed all of it at once. The rule is to always put a series resistor (330 Ω to 1 k Ω) on an LED driven from a logic pin.

11 ESP32 firmware

The ESP32 firmware is split into small modules. `uart_reg` is a register level driver for the ESP32 UART1 and UART2 peripherals, with no Arduino HardwareSerial layer. `flash_link` performs the high level flash operations against the FPGA controller, with retry and verify and a 60 record RAM cache for the dashboard. `fpga_uart_link` is the codebook layer that turns decoded characters into protocol patterns and drives the speaker through the FPGA. `protocol` is the acoustic protocol state machine (presence, retries, abort, registration). `web_link` is the WebSocket client and JSON layer for the dashboard.

11.1 Register level UART

The two UART links to the FPGA are driven by writing the UART peripheral registers directly (the FIFO, CLKDIV, CONF0 and STATUS registers of UART1 and UART2), not through HardwareSerial. The baud divider for 115200 from the 80 MHz APB clock is

$$\text{int} = \left\lfloor \frac{80\,000\,000}{115200} \right\rfloor = 694, \quad \text{frac} = \left\lfloor \frac{80\,000\,000 \times 16}{115200} \right\rfloor \bmod 16 = 7,$$

so the CLKDIV register is $694 | (7 \ll 20) = 0x7002B6$, giving $80\,000\,000 / (694 + 7/16) = 115,208$ baud, 0.007% off. A free running 1 MHz hardware timer (TIMG0) gives microsecond timeouts for the UART reads without using `micros()`.

11.2 Flash link: retry and verify, and a defensive read

When the dashboard sends a note, `web_link` parses the JSON and calls `flash_log_append`, which builds the 16 byte record, sends opcode L plus the bytes over UART1, waits for the K acknowledgement, reads the head back with H, reads the just written slot back with G, and compares all 16 bytes; on a mismatch it retries, up to 5 times. The record is also pushed into the RAM cache so the dashboard still shows it even if a later NOR read misbehaves. The settings save works the same way against the 32 byte sector. The settings read, `flash_get_settings`, validates before returning: the magic byte must be 0xA5, the retry count must be between 1 and 30, and the name must be printable ASCII; if any check fails it returns safe defaults, so the dashboard never shows a garbled name or an absurd count even when the NOR has a half written sector.

12 The acoustic protocol

The protocol uses an alternating bit codebook. A pattern is two or three symbols, with the rule that two consecutive symbols must differ. This makes detection robust, because the decoder triggers on a *change* of tone, which is easy to see, rather than on silence, which is hard to measure over a noisy channel. There are six symbols: lowercase **a**, **b**, **c** on the 2000 Hz master carrier (bit 0, bit 1, end of frame), and uppercase **A**, **B**, **C** on the 2400 Hz slave carrier. The master sends lowercase and the slave sends uppercase, so each side knows who spoke. A few patterns from master to slave: **ab** is a presence request, **ba** assigns an id, **aba** is an abort.

The channel is half duplex, so only one side transmits at a time. The master does carrier sense: it watches the decoder for any uppercase symbol and waits for the channel to be quiet for at least one second before transmitting. Retries are handled by the ESP32: a presence request with no answer within five seconds is repeated up to a configurable number of times (stored in the flash settings), after which the master gives up and tells the dashboard.

13 Power supply

Power enters as a single 12 V supply rated 2 A. A fuse and a series Schottky diode protect against overcurrent and reverse polarity, then an LM2596 based DFRobot DFR1202 buck module steps the 12 V down to 3.3 V at up to 0.5 A for all the digital parts (the FPGA board, the ESP32, the NOR flash, the ADC). Input and output of the buck are decoupled with 100 μ F and 100 nF capacitors. The 12 V rail is kept separate from the buck and feeds the two high current loads directly: the 6 W power LED and the speaker stage, each switched by its own low side MOSFET from a logic pin. Keeping the LED and speaker on the raw 12 V rather than on the 3.3 V buck keeps their switching current off the supply that feeds the sensitive ADC and flash.

14 What worked and what did not

The parts that worked end to end are the bus and its address map, the SDRAM controller once the refresh interval and the 1.54 ns clock phase shift were set, the 512 point FFT with the parabolic peak decoder and its ± 3 neighbourhood, the alternating bit codebook (helped a lot by giving each direction its own short sub codebook for the most repeated message, the presence request), the NOR flash controller after the decoupling and clock work, the WebSocket dashboard, and the register level UART driver on the ESP32.

A few things were tried and dropped. The early design had a generic DMA between the ADC and SDRAM with the CPU programming descriptors; it spent too many CPU cycles per window, so I folded the gather and FFT launch into the streaming accelerator. The ± 1 decoder neighbourhood was too strict and was widened to ± 3 . The plain HardwareSerial on the ESP32 worked but did not show register level work, so I replaced it with the direct register driver.

What still wobbles is mostly the wiring. The W25Q breakout is on Dupont jumpers, so the SPI clock cannot realistically go above 633 kHz; on a proper board with a ground plane it would run far faster with no retries. The decoupling is 1 plus 10 μ F rather than the more orthodox 100 nF plus 10 μ F, because that is what I had on hand. And the very first status register read after power up sometimes returns 0xFF before the chip is ready, which makes the firmware print a false "chip locked" warning that clears on the next read.

15 Conclusions

The master node is a working integration of a custom SoC on the Gowin GW2AR-18, an acoustic receive pipeline from analog microphone through a 12 bit ADC, a 512 point FFT and a tone decoder, a NOR flash store driven over SPI, and an ESP32 that runs my AUSB protocol on top

and bridges it to a web dashboard. The single most useful lesson was that for the mixed signal parts the datasheet number is where the problem starts, not where it ends: the simple relation $\Delta V = It/C$ explained a "flash write succeeds but the data is not there" bug that had looked like a firmware fault for days.

A Quick reference

Table 10: Key numbers used in the design.

Bus / CPU clock	40.5 MHz
SDRAM clock	40.5 MHz; die on phase shifted <code>clkoutp</code> (+1.54 ns, 22.5°)
SDRAM refresh	one AUTO REFRESH every 7.81 μ s
ADC sample rate	46.875 kHz (40.5 MHz / 864)
FFT	512 point, bin width 91.6 Hz
Tone bins	22, 27, 32, 39, 50 (2000/2400/2900/3500/4500 Hz)
NOR SPI clock	633 kHz (divide by 64)
ADC SPI clock	750 kHz (divide by 54)
UART baud	115200 (divide by 352)
W25Q64 JEDEC ID	EF 40 17
Sector erase / page program	up to 400 ms / up to 3 ms
Flash poll timeout	25e6 cycles ($\approx$ 620 ms)
Decoupling at the flash	1 μ F + 10 μ F
Log record / settings	16 B / 32 B, 512 ring slots